Requirements Checklist

- Use an appropriate base image
- Set a working directory where needed
- Copy only required files
- Avoid unnecessary files
- Expose the required application port
- Start the application correctly

Final Folder Structure

```
production-docker/
■■■ Dockerfile
■■■ .dockerignore
■■■ app/
■   ■■■ index.html
■■■ README.md
```

Step 1 — Create the Project Folder

Set up the folder structure exactly as specified in the task.

```
mkdir -p production-docker/app
cd production-docker
touch Dockerfile .dockerignore README.md
touch app/index.html
```

Step 2 — Add the Application File

Create a simple static page in **app/index.html** — this is the only file the container actually needs to serve.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Production Docker Demo</title>
</head>
<body>
  <h1>Hello from the production-style container!</h1>
  <p>This static page is served by Nginx from inside a minimal Docker image.</p
>
</body>
</html>
```

Step 3 — Write the .dockerignore

Exclude files that should never be sent to the Docker build context or baked into the image.

```
.git
.env
*.log
node_modules
__pycache__
temp
```

> Why this matters: everything in the build context is uploaded to the Docker daemon before the build even starts. Excluding secrets, logs, and dependency caches keeps builds fast and prevents sensitive files from ever reaching a layer.

Step 4 — Choose the Base Image

Use **nginx:1.27-alpine** — a small, official, well-maintained image purpose-built for serving static content. Alpine keeps the final image size down compared to a full Debian/Ubuntu base.

```
FROM nginx:1.27-alpine
```

Step 5 — Set the Working Directory

Point WORKDIR at Nginx's default web root so later COPY instructions can use simple relative paths.

```
WORKDIR /usr/share/nginx/html
```

Step 6 — Clear the Default Content

Nginx ships with a default placeholder page. Remove it so only the app's own files are served.

```
RUN rm -rf ./*
```

Step 7 — Copy Only the Required Files

Copy just **app/index.html** — nothing else from the repo (no source control files, configs, or build scripts) ends up in the image.

```
COPY app/index.html .
```

Step 8 — Expose the Application Port

Document the port the app listens on. Nginx serves HTTP on port 80 by default.

```
EXPOSE 80
```

Step 9 — Start the Application Correctly

Run Nginx in the foreground. If Nginx were left to daemonize (its default behavior), the container's main process would exit immediately and Docker would consider the container stopped.

```
CMD ["nginx", "-g", "daemon off;"]
```

Complete Dockerfile

```
# Use a small, official, production-appropriate base image
FROM nginx:1.27-alpine

# Set a working directory (Nginx's default web root)
WORKDIR /usr/share/nginx/html

# Remove the default Nginx placeholder page
RUN rm -rf ./*

# Copy only the required files (static site content)
COPY app/index.html .

# Expose the port the app is served on
EXPOSE 80

# Start the application (Nginx runs in the foreground so the container stays al
ive)
CMD ["nginx", "-g", "daemon off;"]
```

Step 10 — Build the Image

```
docker build -t production-app:v1 .
```

Step 11 — Run the Container

```
docker run -d -p 8080:80 --name production-app production-app:v1
```

Then open **http://localhost:8080** in a browser to confirm the page loads.

Step 12 — Verify

- Confirm the container is running: `docker ps`
- Check logs if needed: `docker logs production-app`
- Inspect image size: `docker images production-app`

Design Choices Summary

Requirement	How it was met
Appropriate base image	nginx:1.27-alpine — small & production-ready
Working directory	/usr/share/nginx/html (Nginx web root)
Copy only required files	Only app/index.html is copied in
Avoid unnecessary files	.dockerignore excludes .git, .env, logs, caches

Expose required port	EXPOSE 80
Start correctly	CMD ["nginx","-g","daemon off;"] runs in foreground